NEXT LEVEL

NEXT LEVEL SOFTWARE DEVELOPMENT

With AI ENGINEERING

Be A Software Developer AI Can't Easily Replace

TS JS ex

TO KNOW MORE

01322-810873
01335-147192

next.programming-hero.com

NEXT LEVEL

PLACEMENT STATISTICS

Our Graduates Are Building Successful Careers
At 200+ Companies Across 26+ Countries

Job Roles:

- Full Stack 56%
- Frontend 30%
- Backend 14%

Employment Type

- 66% Full-Time
- 23% Paid Internship
- 11% Contractual

Work Mode:

- Onsite 54%
- Remote 42%
- Hybrid 4%

TO KNOW MORE
01322-810873
01335-147192

NEXT LEVEL

BOOTCAMP AT A GLANCE

5+	55+	650+	5+
Lerning Missions	Modules	Video Lessons	Hands-On Assignments

TECHNOLOGIES & SKILLS YOU WILL LEARN

Full-Stack Development

TypeScript · Next.js · Node.js · Express.js · NestJS · OOP · Auth · RBAC

Database Management

SQL · PostgreSQL · Drizzle ORM · Redis · Vector Database

AI Engineering

LLM · RAG · LangChain · Multi-Agent Systems · Prompt Engineering · Tool Calling · AI Guardrails · Embeddings

Production & Deployment

Docker · Multi-Container Application · NGINX · AWS · GitHub Actions · CI/CD · VPS

Software Engineering Fundamentals

Multi-tenant Architecture · Database Design Techniques · SaaS Product Development · Clean Code · Industry Best Practices · System Design · Testing · Scalability

TO KNOW MORE
01322-810873
01335-147192

next.programming-hero.com

NEXT LEVEL SOFTWARE DEVELOPMENT WITH AI ENGINEERING

CONTENTS

Mission 01: BUILD WITH TYPESCRIPT

MODULE 01 / TYPESCRIPT FOUNDATIONS

- Why TypeScript exists: the problem with plain JavaScript
- TypeScript setup and tsconfig.json
- Compiling and running TypeScript
- Basic data types
- Arrays and objects
- Functions, parameters, return types
- Optional and default parameters
- Literal types
- Spread and rest operators
- Destructuring
- Type alias
- Union and intersection types
- Ternary, nullish coalescing, optional chaining
- null, undefined, unknown, never

MODULE 02 / ADVANCED TYPESCRIPT TYPES

- Type assertion
- Interfaces, and interface vs type alias
- Generics – the mental model
- Generics with interfaces
- Generics with functions
- Constraints in TypeScript
- keyof constraint with generics
- Enum
- as const instead of Enum
- Conditional types
- Mapped types
- Utility types (Partial, Pick, Omit, Record, Required, Readonly)

MODULE 03 / OBJECT-ORIENTED PROGRAMMING IN TYPESCRIPT

- Classes and objects
- Constructors and parameter properties
- Inheritance (1st pillar of OOP)
- Type guards using typeof and in
- Type guards using instanceof
- Access modifiers (public, private, protected)
- Getters and setters
- Static properties and methods
- Polymorphism (2nd pillar of OOP)
- Abstraction (3rd pillar of OOP)
- Encapsulation (4th pillar of OOP)
- Mission recap and common beginner mistakes

MODULE 04 / ASSIGNMENT 1

- Solve real programming problems using TypeScript
- Type-safe utility functions and generic helpers
- A small OOP-modelled domain (e.g. library / vehicle / student system)
- Submission rules, deadline policy, and how assignments are graded

Conceptual sessions (2):

1. “How the JS runtime actually runs your code”
2. “Thinking in types: reading and designing type signatures”

Mission 02: BUILD SCALABLE BACKENDS

MODULE 05 / NODE.JS RUNTIME & ARCHITECTURE

- How the web works: client, server, HTTP, request/response cycle
- Understanding the Node.js runtime
- Event loop & non-blocking architecture
- Synchronous vs asynchronous programming
- Callbacks and callback hell
- Promises
- Async / await and error handling with try...catch
- CommonJS vs ES Modules
- npm ecosystem & package management
- package.json, scripts, dependencies vs devDependencies

MODULE 06 / NODE.JS CORE MODULES & YOUR FIRST SERVER

- Node.js core modules: fs, path, http, process, os
- Reading and writing files (sync vs async)
- Streams & buffers fundamentals
- Environment variables & configuration with dotenv
- Building your first HTTP server without a framework
- Routing by hand – and why it becomes painful
- Debugging Node.js applications
- Nodemon / tsx and the TypeScript + Node setup
- Module summary & best practices

MODULE 07 / EXPRESS FUNDAMENTALS & YOUR FIRST REST API

- Setting up an Express + TypeScript project
- Understanding req, res & next
- Express routing fundamentals
- Route parameters, query strings, request body
- Middleware: built-in, third-party, custom
- API design & REST principles
- Status codes and consistent response shapes
- Building an in-memory CRUD API
- API testing using Postman / Bruno
- Module summary

MODULE 08 / POSTGRESQL ESSENTIALS & CONNECTING EXPRESS WITH PG

- What a relational database is (tables, rows, columns)
- Installing PostgreSQL and pgAdmin
- Creating a database & table, common data types
- INSERT, SELECT, WHERE, ORDER BY
- UPDATE, DELETE
- LIMIT & OFFSET
- Primary key & foreign key (practical intro)
- Connecting PostgreSQL with the pg Pool
- Parameterized queries and why they prevent SQL injection
- Rebuilding the in-memory CRUD API against a real database
- Practice exercises

MODULE 09 / SCALABLE BACKEND ARCHITECTURE & CLEAN CODE

- Why project structure matters
- Separation of concerns
- SOLID principles applied in functional-style TypeScript
- Clean code practices
- Feature-based (modular) architecture
- Layered architecture: route → controller → service → repository
- Config layer and utility layer
- Refactoring the CRUD API into layers
- Centralized error handling & custom error classes
- Request validation with Zod
- Production-ready folder structure
- Basic API documentation

MODULE 10 / ASSIGNMENT 2 (SMALL BACKEND PROJECT)

- Build a layered, modular REST API on Express + TypeScript + PostgreSQL (pg)
- Full CRUD with Zod validation and consistent error responses
- Basic search, filtering & pagination
- Production-ready folder structure
- README + API documentation

Conceptual sessions (2 small backend projects):

1. Project 1 – Raw Node.js server (after Module 06): a file-based Notes / Todo API using only http, fs and path – no framework
2. Project 2 – Express + PostgreSQL mini API (after Module 09): a layered Expense Tracker / Book Catalogue API with validation and centralized error handling

MISSION 03: DESIGN & OPTIMIZE DATABASES

MODULE 11 / DATABASE ESSENTIALS & RELATIONAL MODEL

- Understanding Data, Information and Database
- What is DBMS and Why
- Different Types of Database Model and the Relational Model
- Anatomy of a Table / Relation
- What is a Key and Super Key
- Candidate, Primary, Alternate and Composite Key
- Explaining Foreign Keys
- Techniques to Design a Database
- Steps of the Top-down Technique
- Relationship and Relationship Cardinality
- Tooling for ER Diagrams and Creating the First ER Diagram

MODULE 12 / DELVING DEEPER: DATABASE NORMALIZATION, POSTGRES INSTALLATION AND APPLICATION INSIGHTS

- Understanding Anomalies
- Understanding Functional Dependency
- Normalization and 1st Normal Form (1NF)
- 2nd Normal Form and Partial Dependency (2NF)
- 3rd Normal Form and Transitive Dependency (3NF)
- Resolving Many-to-Many with a Junction Table
- Completing the ER Diagram
- What is Postgres and Installing Postgres
- Exploring Data Flow in an Application and Exploring PSQL
- Module Summary and Practice Case Study

MODULE 13 / POSTGRESQL ESSENTIALS: CASE STUDY SOLUTIONS, DATABASE MANAGEMENT AND DATA TYPES

- Solving the First Case Study
- Exploring PSQL and its Default Behaviour, Creating a Database
- User, Role and Privilege Management in Postgres
- Granting and Revoking Privileges
- Structured Query Language (SQL)
- Exploring pgAdmin and Valentina Studio
- Create, Update and Delete Database and Data Types in Postgres
- Creating a Table with Multiple Columns and Different Data Types
- Creating a Table with Multiple Columns and Column Constraints
- Different Methods to Define Constraints
- Inserting Data into a Table and Checking Constraints

MODULE 14 / POSTGRESQL FUNDAMENTALS AND ADVANCED DATA MANIPULATION TECHNIQUES

- Using the ALTER Keyword to Modify Tables and Setting Up PostgreSQL in VS Code
- Expanding on the ALTER Keyword for Table Modification
- Different Methods to Alter Tables for Primary Key, Unique etc.
- Mastering SELECT Queries: Column Aliasing and Result Ordering
- Data Filtering: WHERE Clause, Logical Operators and Comparison Operators
- Exploring Scalar and Aggregate Functions
- Logical Negation NOT, Understanding NULL and the Null-Coalescing Operator
- Exploring IN, BETWEEN, LIKE and ILIKE Operators
- Pagination with LIMIT / OFFSET and Data Deletion
- Understanding and Using the UPDATE Operator

MODULE 15 / MANAGING DATES, RELATIONSHIPS AND UNDERSTANDING JOIN IN POSTGRESQL

- Handling Date and Date Functions
- Grouping and Filtering Data with GROUP BY and HAVING
- Constructing Relationships with Foreign Key Constraints
- Enforcing Referential Integrity: Behaviors During Data Insertion
- Enforcing Referential Integrity: Behaviors During Data Deletion
- Joining Tables with Inner Join
- Understanding Left and Right Joins
- Exploring Full, Cross and Natural Joins
- Query Practice: Part 1
- Query Practice: Part 2

MODULE 16 / ADVANCED QUERY TECHNIQUES AND DATABASE OBJECTS IN POSTGRESQL

- Query Practice: Part 3
- Exploring Subqueries
- Utilizing Subqueries in Different Clauses
- Exploring Views
- Exploring Functions
- Exploring Stored Procedures
- Practical Implementation of Triggers
- Indexing Techniques
- Understanding How Indexing Works in PostgreSQL

MODULE 17 / ASSIGNMENT 3 – DESIGN & BUILD A PRODUCTION DATABASE

- Requirement analysis & entity identification from a written brief
- ER diagram and relationship modeling
- Normalization up to 3NF
- Primary, foreign & composite keys; referential integrity & cascading rules
- Schema + seed data in PostgreSQL
- Query set: joins, aggregations, subqueries / CTEs
- At least one function and one trigger
- Indexing strategy with EXPLAIN evidence
- Schema documentation

Conceptual sessions (2):

1. “Normalize this live: turning a messy spreadsheet into 3NF”
2. “Index or not? Reading EXPLAIN ANALYZE together

MISSION 4: BUILD COMPLETE SAAS PRODUCT

PART A / DRIZZLE BASICS & SMALL SAAS APPLICATION BACKEND WITH NODE-EXPRESS

MODULE 18 / ORM ESSENTIALS (DRIZZLE)

- Why ORMs exist and what problem they solve (ORM vs query builder vs raw SQL)
- Installing and initializing Drizzle with PostgreSQL
- Drizzle Kit & CLI
- Drizzle schema anatomy
- Tables, columns, data types and constraints
- Migrations: generate, run and push
- Type-safe queries and inferred types ($inferSelect, $inferInsert)
- Drizzle Studio
- Seeding the database
- Basic CRUD operations with Drizzle

MODULE 19 / RELATIONS & ADVANCED QUERY TECHNIQUES

- Defining one-to-one, one-to-many, many-to-many relations
- Relational queries API (db.query with with)
- SQL-like API: select, joins, partial selects
- Filtering operators (eq, and, or, like, inArray ...)
- Sorting
- Pagination (offset & cursor)
- Aggregations and grouping
- Transactions with Drizzle
- The sql template for raw queries when you need them
- Migrating the Mission 2 repository layer from pg to Drizzle

MODULE 20 / SAAS SYSTEM DESIGN & MULTI-TENANT ARCHITECTURE

- Product requirement analysis for the course SaaS
- SaaS architecture fundamentals
- Multi-tenant architecture: the three isolation models (DB per tenant, schema per tenant, shared schema)
- Workspace & team design
- Data isolation strategies and the tenant-scoping rule
- High-level system design of the product
- Designing the full Drizzle schema for the SaaS
- Drawing the ERD before writing a line of code

MODULE 21 / PROJECT SETUP, DATA MODELING & REQUEST VALIDATION

- Bootstrapping the production backend repository
- Folder structure, config layer and app bootstrap
- Implementing the Drizzle schema and first migration
- Global error handler and custom AppError
- Request validation with Zod schemas per route
- catchAsync / async error wrapper
- Consistent API response envelope
- Logging setup and environment separation

MODULE 22: AUTHENTICATION: REGISTRATION, LOGIN & JWT

- Authentication vs authorization
- Password hashing with bcrypt
- Registration flow
- Login flow

- JWT theory: header, payload, signature
- Access token generation and verification
- Storing tokens: cookies vs headers
- Common auth mistakes and how to avoid them

MODULE 23: AUTHORIZATION: MIDDLEWARE, PROTECTED ROUTES & RBAC

- Authentication middleware
- Protected routes
- Role-based access control (RBAC)
- Refresh tokens & refresh token rotation
- User status management (active, blocked, deleted)
- Change password, forgot password, reset password
- Testing auth flows in Postman / Bruno

MODULE 24: WORKSPACE, TEAM & MEMBERSHIP MANAGEMENT

- Workspace & team architecture
- Creating and switching workspaces
- Membership & invitation system (invite, accept, revoke)
- Workspace-level roles (owner, admin, member)
- Workspace authorization guard
- Enforcing tenant isolation on every query
- Ownership transfer and member removal edge cases

MODULE 25: DATA & FILE MANAGEMENT

- Modeling core product entities with Drizzle (projects, tasks, comments)
- File upload with Multer
- Cloud storage integration (Cloudinary / S3)
- Secure file access and signed URLs
- Metadata management
- File size, type and quota validation
- Production data modeling patterns (soft delete, audit fields)

MODULE 26: SEARCH, FILTERING, SORTING & PAGINATION

- Search across multiple fields
- Dynamic filtering with a reusable query builder
- Sorting and field selection
- Pagination strategies at API level
- Query optimization with Drizzle
- Indexing the queries the product actually runs
- Performance best practices

MODULE 27: PRODUCTION BACKEND ENGINEERING

- Transactions for multi-step operations
- Analytics & reporting endpoints (aggregations)
- Logging & monitoring
- Structured error handling in production

- Rate limiting and basic security (helmet, CORS, sanitization)
- Production reliability patterns
- API documentation (Postman collection / Swagger)
- Part A recap

MODULE 28 / ASSIGNMENT 4 – BUILD A SAAS BACKEND

- Multi-tenant architecture with workspace isolation
- Authentication & authorization with RBAC
- Team & workspace management with invitations
- File management
- Search, filtering & pagination
- Analytics endpoints
- Modular + layered architecture, validation, error handling
- API documentation and deployed API link

Conceptual sessions (3 – together they build 1 medium backend project)

1. Schema & setup: requirements, ERD and Drizzle schema for a small multi-tenant app (e.g. helpdesk / ticketing)
2. Auth & tenancy: JWT auth, RBAC and workspace-scoped queries
3. Features & finish: CRUD, search / filter / pagination, error handling, API docs

PART B / NEXT.JS BASICS & SMALL SAAS APPLICATION FRONTEND WITH NEXT.JS

MODULE 29 / NEXT.JS FUNDAMENTALS & THE APP ROUTER

- Why Next.js: React with a framework around it
- Project setup and folder conventions
- App Router: pages, layouts, nested routes
- Route groups, dynamic routes, catch-all routes
- loading.tsx, error.tsx, not-found.tsx
- Metadata & SEO basics
- Linking, navigation and router hooks
- Styling with Tailwind CSS + shadcn/ui setup

MODULE 30 / SERVER COMPONENTS, DATA FETCHING & SERVER ACTIONS

- Server vs client components: the decision rule
- Data fetching strategies (static, dynamic, streaming)
- Caching and revalidation
- Route handlers (API routes) in Next.js
- Server Actions: forms without an API layer
- Suspense and streaming UI
- Connecting the Next.js app to the Part A backend

MODULE 31 / AUTHENTICATION ON THE FRONTEND

- Login and registration UI
- Storing and refreshing tokens safely
- Session handling with HTTP-only cookies
- Next.js middleware for route protection
- Protected layouts and redirect logic
- Role-based UI rendering
- Workspace switching in the UI

MODULE 32 / BUILDING THE PRODUCT DASHBOARD: LAYOUT & UI ARCHITECTURE

- Dashboard UI architecture
- Sidebar, topbar and nested dashboard layouts
- Responsive UI and mobile navigation
- Component composition patterns
- Loading skeletons and empty states
- Toasts, modals and confirmation dialogs
- Dark mode

MODULE 33 / FORMS, VALIDATION & FILE UPLOAD

- Forms with React Hook Form
- Schema validation with Zod (sharing schemas with the backend)
- Field-level and form-level error handling
- Multi-step forms
- File upload UI with preview and progress
- Optimistic UI updates
- Handling server validation errors gracefully

MODULE 34 / DATA TABLES, SEARCH, FILTERING & PAGINATION

- Building a reusable data table (TanStack Table)
- Server-side pagination wired to the backend
- Search with debouncing
- Filters and sorting driven by URL search params
- Row actions and bulk actions
- Shareable, bookmarkable filtered views

MODULE 35 / CHARTS, ANALYTICS & REPORTING UI

- Charts & data visualization (Recharts)
- Building the analytics dashboard
- Stat cards and trend indicators
- Date-range filtering
- Exporting reports (CSV / PDF)
- Making dashboards fast: what to render on the server

MODULE 36 / SCALABLE FRONTEND ARCHITECTURE & PERFORMANCE

- Reusable component systems and design tokens
- State management (Context, Zustand or RTK – when each is right)
- Server state vs client state (TanStack Query)
- Performance: images, fonts, bundle size, dynamic imports
- Frontend project structure at scale
- Error boundaries and production logging
- Environment configuration and deploying to Vercel
- Production best practices

MODULE 37 / ASSIGNMENT 5 – BUILD A SAAS FRONTEND

- Full frontend for the Assignment 4 backend
- Authentication, protected routes, role-based UI
- Team & workspace management screens
- File management UI
- Data tables with search, filtering & pagination
- Analytics dashboard
- Responsive, production-ready UI
- Deployed live link + GitHub repo

Conceptual sessions (3 – together they build 1 medium frontend project, the UI for the Part A conceptual backend)

1. App shell: App Router layouts, auth pages, protected routes
2. Data screens: tables with search / filter / pagination, forms with validation
3. Dashboard & ship: charts, role-based UI, deploy to Vercel

MISSION 05: SHIP SOFTWARE TO PRODUCTION

MODULE 38 / INTRODUCTION TO DOCKER

- Get Started with Docker
- What is Docker & Why?
- Virtual Machines vs Docker Containers
- Where We Use Docker
- How Docker Works on Different OSes

- Docker Installation and Setup (Windows)
- Docker Installation and Setup (Mac)
- Docker Installation and Setup (Linux)
- Configuring VS Code & Creating the First Container

MODULE 39: DOCKER IMAGE AND CONTAINER

- What are Images & Containers
- Understanding Image Types and Using Pre-built Images
- Writing Our First Dockerfile
- Building an Image & Running a Container Based on Our Image
- Deep Dive into Docker Images
- Managing Images and Practising the Most-used Commands
- Managing Containers and Practising the Most-used Commands
- Docker Container Restart Policies
- Pushing a Docker Image to Docker Hub
- Pulling a Docker Image from Docker Hub

MODULE 40: DOCKER VOLUMES AND DATA MANAGEMENT

- Why Data Management Matters in Docker
- Understanding Data Categories
- The Writable Container Layer Explained
- Understanding Docker Volumes
- Managing Volumes with the Docker CLI
- Hands-on Docker Bind Mounts
- Solving a Classic Problem by Combining Volumes and Bind Mounts
- Environment Variables and .env Files
- The .dockerignore File
- Module Recap and Volumes vs Bind Mounts Decision Guide

MODULE 41 /DOCKER NETWORKING AND CROSS-CONTAINER COMMUNICATION

- Container Communication and Types of Communication
- Container to WWW Communication
- Container to Local Host Machine Communication
- Container to Container Communication
- How Docker Networking Simulates a Real-World Network Internally
- Creating a Container and Communicating with the Web (WWW)
- Implementing Container to Host Communication
- Container-to-Container Communication: IP Address vs Docker DNS
- Docker Architecture Overview
- Module Recap and Capstone Project Idea

MODULE 42 / BUILDING MULTI-CONTAINER APPLICATIONS

- Introduction
- Steps to Build a Multi-Container Application
- Analyzing the App Codebase
- Dockerizing the PostgreSQL Service
- Dockerizing the Node.js Application
- Dockerizing the Next.js Application
- Running the Dockerized Next.js App with a Bind Mount
- Adding a Security Layer for the PostgreSQL Container
- Module Summary

MODULE 43 / MULTI-CONTAINER ORCHESTRATION WITH DOCKER COMPOSE AND MAKEFILE AUTOMATION

- Introduction to Docker Compose
- Creating the First Docker Compose File
- Creating a Service for the PostgreSQL Container
- Creating a Service for the Node-Express Container
- Creating a Service for the Next.js Frontend Container
- Module Summary

MODULE 44 / VPS DEPLOYMENT USING DOCKER, NGINX AND CI/CD SETUP USING GITHUB ACTIONS

- Introduction to Reverse Proxy
- Reverse Proxy Hands-on
- Understanding Load Balancers and Hands-on Implementation
- Setting Up the VPS Environment and Installing Docker
- How to Write a Dockerfile Smartly
- Writing Docker Compose and Nginx Config Files for Production
- Deep Dive into CI/CD: Building GitHub Actions Workflows
- Setting Up Env and Upgrading Nginx on the VPS
- CI/CD Pipeline Recap & Testing

Conceptual sessions (2)

1. “From localhost to a live domain: tracing a request through DNS → NGINX → containers”
2. “It broke in production: reading logs and fixing a failed deploy together”

MISSION 6: BUILD AI-POWERED SYSTEMS

PART A / LLM, LANGCHAIN, VECTOR DB, RAG & AGENTS

MODULE 45 / LLM APPLICATION DEVELOPMENT FUNDAMENTALS

- How large language models work (intuition, no math)
- Tokens, context windows and cost
- OpenRouter API (and provider-agnostic thinking)
- Prompt engineering fundamentals
- System prompts
- Few-shot prompting and prompt patterns
- Common failure modes: hallucination, drift, prompt injection

MODULE 46 / BUILDING A PRODUCTION CHAT API

- Structured output
- JSON mode and schema-constrained responses
- Streaming responses to the client
- Token budgeting and truncation strategies
- Retries, timeouts and graceful degradation

MODULE 47: LANGCHAIN APPLICATION DEVELOPMENT

- Why LangChain exists (and when not to use it)
- Prompt templates
- Output parsers
- Chains
- LCEL (LangChain Expression Language)
- Runnables and composition
- Message history
- Debugging chains

MODULE 48: DOCUMENT PROCESSING & EMBEDDINGS

- Document loaders (PDF, web, text, CSV)
- Text splitters
- Chunking strategies and why chunk size matters
- What are embeddings
- Embedding models and dimensions
- Vector stores
- Retrievers
- Building your first AI pipeline end-to-end

MODULE 49: VECTOR DATABASES: PGVECTOR

- What a vector database actually does
- Setting up pgvector on PostgreSQL
- Alternative vector databases
- Storing, indexing and querying vectors

- Semantic search
- Hybrid search (keyword + semantic)
- Metadata filtering
- Choosing a vector store for production

MODULE 50: BUILDING A PRODUCTION RAG SYSTEM

- RAG architecture end-to-end
- Retrieval strategies (top-k, re-ranking)
- Context injection and prompt assembly
- Citations and source attribution
- Handling “I don’t know” correctly
- Evaluation: measuring whether your RAG is any good
- Building a PDF knowledge-base assistant
- Cost and latency optimization

MODULE 51: LANGGRAPH & AI AGENTS

- Why LangGraph – limits of linear chains
- Graph state management
- Nodes & edges
- Conditional routing
- Multi-step AI workflows
- Tool calling
- Function calling and schema design
- Debugging and tracing agent runs

MODULE 52 / ADVANCED AGENT SYSTEMS

- Agent memory (short-term and long-term)
- Human-in-the-loop checkpoints
- Building multi-agent systems
- Agent specialization and task delegation
- Guardrails and safety checks
- Production agent architecture
- AI-powered search and recommendation features
- Email / task assistant patterns

MODULE 53 / ASSIGNMENT 6 – BUILD A PRODUCTION AI APPLICATION

- LangChain pipeline
- LangGraph agent with tool calling
- RAG system over user-uploaded PDFs
- pgvector
- Streaming chat interface
- Evaluation notes and cost analysis

Conceptual sessions (3)

1. “Prompt engineering that survives production”
2. “RAG in the real world: what breaks at scale”
3. “Designing agents that don’t loop forever”

PART B / INTEGRATING THE AI SERVICE WITH THE PRODUCT

MODULE 54 / SERVICE-TO-SERVICE COMMUNICATION

- Internal API design and shared types
- Service-to-service authentication
- Timeouts, retries and circuit-breaking basics
- Error propagation across services

MODULE 55 / API GATEWAY & REQUEST ROUTING

- The API gateway pattern
- Routing requests across backend and AI services
- Authentication & tenant context propagation
- Request / response transformation
- Centralized rate limiting
- Centralized logging and request tracing
- Versioning your APIs

MODULE 56 / STREAMING, BACKGROUND WORKERS & CACHING

- Background workers and job queues
- Long-running AI jobs and status polling
- Caching with Redis
- Rate limiting AI endpoints per workspace
- Usage tracking, quotas and cost optimization
- Production AI patterns and scaling AI services

MODULE 57 / FRONTEND INTEGRATION: AI FEATURES IN NEXT.JS

- Consuming streaming responses in the browser
- Building the chat interface with streaming UI
- Uploading documents to the knowledge base
- Displaying citations and sources
- Loading, error and retry states for AI calls
- Optimistic UI for AI actions

MODULE 58 / FRONTEND INTEGRATION: AGENT DASHBOARD & POLISH

- Knowledge-base management UI
- Agent run history and traces in the dashboard
- AI usage analytics per workspace
- Feature gating and quota UI
- Accessibility and mobile polish

LIVE

AGENTIC AI MASTERCLASS

1. Introduction to Agentic Coding
2. Setting Up Your AI Coding Environment (Claude Code & OpenCode)
3. Context Engineering & AI Memory (AGENT.md & CLAUDE.md)
4. AI Development Workflow
5. Specification-Driven Development (SDD)
6. Advanced Agent Workflows
7. MCP (Model Context Protocol)
8. Engineering Automation
9. Production Demo
10. Engineering Best Practices

REWARD COURSES

Reward Course 01 / SYSTEM DESIGN

- What Is System Design?
- System Design vs. Software Architecture
- Monolithic vs. Microservices Architecture
- Functional vs. Non-Functional Requirements in System Design
- CAP vs. PACELC Theorem in Distributed Systems
- Availability and Fault Tolerance in Distributed Systems
- Retry Mechanisms, Backoff Strategies, and Idempotency
- Observability in Distributed Systems: Metrics, Logs, and Traces
- API Architectural Styles: REST, GraphQL, RPC
- SQL, NoSQL, and NewSQL Databases in System Design
- Database Partitioning and Sharding in System Design
- System Design Trade-Offs and more topics

Reward Course 02 / NESTJS BACKEND

- Introduction to NestJS
- Why NestJS?
- Installing & Creating a NestJS Project
- Project Structure & Architecture

- Modules, Controllers & Services
- Dependency Injection
- Providers & Custom Providers
- Routing & Request Handling
- Middleware, Pipes, Guards & Interceptors
- Exception Filters & Error Handling
- Request Validation with class-validator
- Configuration & Environment Variables
- Authentication with JWT (Overview)
- Database Integration with Prisma (Overview)
- NestJS Best Practices

Reward Course 03 / TESTING & QUALITY

- Testing Fundamentals
- Unit Testing with Vitest
- Integration Testing
- API Testing with Supertest
- Mocking & Test Doubles
- Test-Driven Development (TDD) Basics
- Debugging Strategies
- Test Coverage & Best Practices
- Writing Maintainable Test Suites
- Production Testing Workflow

Reward Course 04 / AWS DEPLOYMENT

- Introduction to Cloud Computing
- AWS Global Infrastructure
- Identity & Access Management (IAM)
- Amazon S3 Object Storage
- Linux Essentials for Cloud Servers
- EC2 Instance Management
- Domain & DNS Configuration
- SSL/HTTPS with Let's Encrypt
- Monitoring & Server Maintenance
- AWS Best Practices

SPECIAL REWARD

SOFTECH NEXUS - A PREMIUM PLACEMENT PROGRAM

- Build essential "Soft Skills" to become industry ready
- Prepare for real interviews through personalized mock sessions
- Receive expert-led professional training for success.
- Improve your English proficiency through English Club and more

BE THE SOFTWARE DEVELOPER AI CAN'T EASILY REPLACE

A Complete 6-Month Bootcamp With 6 Types Of Dedicated Support.Alongside Daily Live Support,You'll Build 10+ Hands-On Practice Projects And An Advanced Full-Stack Multi-Tenant AI SaaS Product. Plus, You'll Get Free Premium Career Support

CONNECT WITH US

Facebook LinkedIn Youtube Instagram

CONTACT

level2@programming-hero.com

+8801322-810873,+8801335-147192

next.programming-hero.com

Interested In Next Level **Batch-8**?
Join Our Facebook Preparation Community

Facebook Group (Click Here)